# What Sparse Autoencoders Miss Is Task-Specific

Mapping the blind spot across tasks and Indian languages

PRISM: course project proposal · October 2026

**In short.** Researchers read a language model's internal signal with *sparse autoencoders* (SAEs), which split it into a few readable features plus leftovers they cannot explain. We started from the popular idea that what a model keeps track of hides in those leftovers. A pilot study on gemma-2-2b says: mostly not, since the readable features carry 74% to 100% of it. But where an SAE *does* miss something, the missing piece is **specific to the task**: a few hidden signals in the leftovers that signals from other tasks, ordinary text or a 16× bigger SAE do not recover. We propose to map this effect **across more tasks** and **across Indian languages**.

## 1 What: the problem and our claim

Language models keep track of things while they read: a running total, which object is in which box, which value a variable holds. To see this internal state, researchers use **sparse autoencoders** (SAEs) [1, 3]. An SAE rewrites the model's internal signal $x$ at one layer as $x = \hat{x} + \varepsilon$: $\hat{x}$ is built from a few human-readable **features**, and $\varepsilon$ is the **error**, the leftovers it cannot explain (sometimes called "dark matter" [6]).

**Our claim.** *When an SAE misses part of what a model is tracking, the missing piece is task-specific*: it lives in a few directions of the error that belong to that task and are not recovered by signals learned from other tasks, from ordinary text, or by a much bigger SAE. The pilot (Section 3, Appendix B) supports this for two of four tasks. This project tests how far it reaches.

- **Q1. Across tasks.** Which kinds of tasks leave a gap, how large is it, and is the missing piece always task-specific? Is it tied to one prompt template, or shared by tasks that need the same computation?
- **Q2. Mechanism.** Is the missing piece in linking tasks the model's *binding* signal, the way it ties an object to its box [7]? Does changing only that signal change the answer?
- **Q3. Across languages.** Do SAEs miss more in Hindi and Hinglish than in English, and is it the same missing piece?

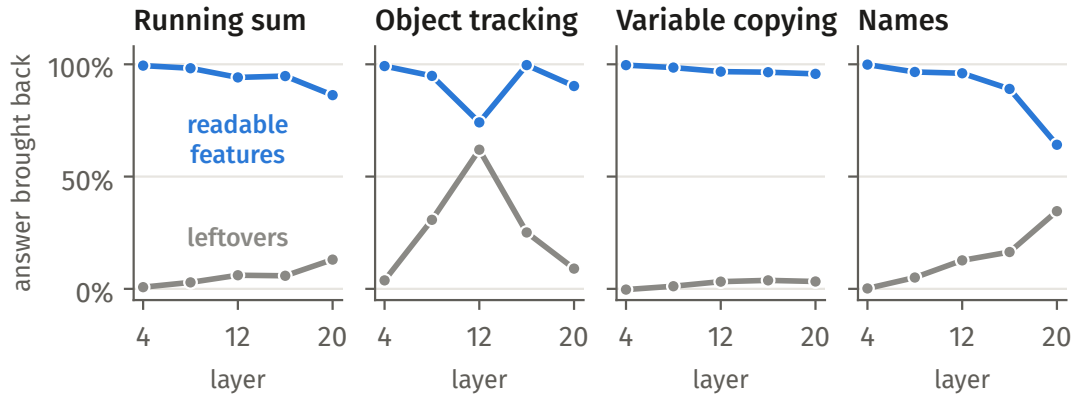
**Expected outcomes:** a map of where SAEs lose task information (which tasks, layers, SAE sizes and languages); a causal test of whether the lost piece is binding; to our knowledge, the first per-language causal audit of how much task information lies in SAE features versus SAE error; and a testing protocol that avoids three traps we found.

## 2 Why it matters

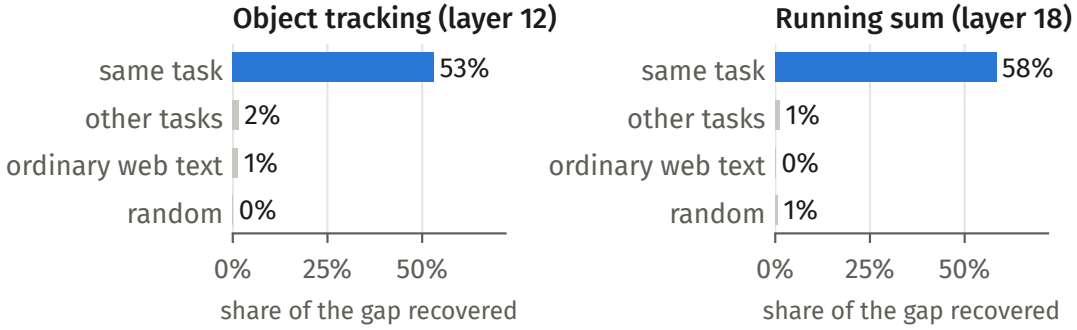
SAEs are now the main tool for opening up language models, and public SAE suites such as Gemma Scope [10] are used to explain, audit and steer model behaviour. The error term is known to matter: it carries weight inside circuits [11] and is partly predictable [6]. If what SAEs miss were *generic*, a bigger or better SAE would fix it. If it is *task-specific*, as the pilot suggests, then an SAE that looks excellent on ordinary text (the ones we use recover 98% to 99% of the model's language-modelling ability) can still be blind to exactly the computation an audit cares about, and SAE-based explanations need a per-task check. The same worry applies to language: the Gemma Scope SAEs were trained on mostly English text [10]. Recent work finds language-specific and shared SAE features across languages [2, 4], but does not measure what the error carries in each language. Users of Hindi and other Indian languages are the people for whom SAE-based safety claims are least tested.

## 3 Pilot study: what we found so far

**Setup** (details in Appendix B). Model: gemma-2-2b [8]. SAEs: Gemma Scope, 16k to 262k features. Four puzzles: a running sum, object tracking ("The key is in box B … The key moves to box C … The key is in box" → C), variable copying, and a well-studied names task as a control [14]. Each comes as pairs of prompts that differ in one step, so the right answer differs; 150 test pairs per setting. **The swap test:** run the changed prompt, but at one layer swap in either the readable part $\hat{x}$ or




**(a)** Swapping the readable features brings back most of the answer; the leftovers bring back little.




**(b)** Where the SAE misses something, only signals from the same task fill the gap (best of each source).

**Figure 1:** The two pilot results behind our claim (150 test pairs; 16k SAEs). Full figures in Appendix B.

the leftovers $\varepsilon$ from the original prompt, and measure how much of the right answer comes back (0% to 100%; a full swap gives 100%). We began with seven puzzles and an original hypothesis that the leftovers carry the tracked state (Appendix A); the model could not solve three of the puzzles, and the hypothesis did not survive.

1. **Readable features carry most of it.** Swapping $\hat{x}$ brings back 74% to 100% of the answer for the three tracking puzzles at every layer tested (4 to 20); the leftovers alone bring back 0% to 13%, except object tracking at layers 8 to 16 (Figure 1a).
2. **One clear gap.** For object tracking at layer 12 the SAE misses 26% of the answer. A single direction taken from the leftovers brings it from 74% to 88%, about half the gap; a random direction adds nothing.
3. **The gap is task-specific.** Directions learned from the same task recover 39% to 53% of it; directions from other tasks 0% to 2%, from ordinary web text 1% (Figure 1b). The running sum shows the same pattern at layer 18 (58% vs 0% to 1%).
4. **A bigger SAE does not remove it.** From 16k to 262k features the object-tracking gap goes $26\% \to 20\% \to 24\% \to 23\% \to 15\%$, and one hidden direction still recovers about half of it at every size.
5. **Three traps**, each of which gave a false result: swapping only the last word (it misses state still held on earlier words); probes that are fooled by the prompt's words; and confusing "readable" with "used" (Appendix B.5).

**Limits.** One model, one SAE family, made-up puzzles; the names control's late-layer loss is spread over about 32 directions, so not every loss is concentrated; the bigger SAEs also use more features per word (Appendix B.7).

## 4 How: plan

**Q1, across tasks.** We add tasks grouped by the computation they need: *linking* (objects in boxes; people and their cities, e.g. "Asha lives in Pune. Ravi lives in Delhi. Asha lives in"), *keeping a count* (longer running sums), *following references* (variable copying) and the names control. For every task,

layer and SAE size we measure the gap (1− share brought back by $\hat{x}$), how many error directions recover half of it, and a **specificity score**: the share recovered by directions from the same task minus the best share from another source (other tasks, web text, random). Transfer between two linking tasks tells us whether the missing piece belongs to a prompt template or to a computation. A sparsity sweep separates SAE size from the number of active features. New prompts are illustrative until checked against the tokenizer and the model's accuracy.

**Q2, binding test.** We compare the missing direction with the directions that carry binding information [7], and edit the signal only along it: if it is binding, the model should name the other object's box.

**Q3, across languages.** Same puzzles, translated, in priority order: **Hindi**, **Hinglish** (Hindi in Latin script, which separates language from script), Tamil or Bengali, and French as a non-Indian control. Illustrative prompts, *to be validated by a native speaker*: शुरुआत 2 से करें। 3 जोड़ें। 1 जोड़ें। 2 जोड़ें। कुल = → 8, and `Jab Rahul aur Priya market gaye, Priya ne ek book di. Book kisko mili? Answer:` → Rahul. Per language we measure (M1) how well the SAE explains ordinary text, from IndicCorp v2 [5] and FineWeb-2 [12]; (M2) the gap and its specificity on the puzzles; (M3) whether the direction learned in English recovers the Hindi gap. A language the model cannot solve is reported as such. Full design in Appendix C.

**Resources and risks.** Everything runs on one 8 GB laptop GPU with the tested pipeline from the pilot (the three main pilot runs took 6.3 hours). Risks: the model may fail the puzzles in Tamil or Bengali (we keep Hindi and Hinglish); answers must be single tokens (we fall back to letters or digits); the GPU limits us to gemma-2-2b.

## 5 When: timeline

| When | Step | Output |
|---|---|---|
| Oct 19–26 (done) | Pilot study, four stages (Appendix B.2) | Original idea rejected; task-specific gap found |
| Week 1 (Oct 27–Nov 2) | Binding test (Q2); sparsity sweep | English mechanism result |
| Weeks 1–2 (to Nov 9) | New linking and counting tasks; gap and specificity per task (Q1) | Task map |
| Week 2 (Nov 3–9) | Hindi and Hinglish puzzles; can the model solve them? | Usable languages |
| Weeks 3–4 (Nov 10–23) | M1 to M3 per language; Tamil or Bengali if usable (Q3) | Language map |
| Week 5 (Nov 24–30) | More test pairs; robustness checks | Final figures |
| Week 6 (Dec 1–7) | Report and final presentation | Submission |

## References


[1] Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Zac Hatfield-Dodds, Alex Tamkin, Karina Nguyen, Brayden McLean, Josiah E. Burke, Tristan Hume, Shan Carter, Tom Henighan, and Christopher Olah. Towards monosemanticity: Decomposing language models with dictionary learning. Transformer Circuits Thread, 2023. URL `https://transformer-circuits.pub/2023/monosemantic-features/index.html`. Published October 4, 2023.

[2] Jannik Brinkmann, Chris Wendler, Christian Bartelt, and Aaron Mueller. Large language models share representations of latent grammatical concepts across typologically diverse languages. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 6131–6150, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.naacl-long.312. URL `https://aclanthology.org/2025.naacl-long.312/`.

[3] Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. *arXiv preprint arXiv:2309.08600*, 2023. URL `https://arxiv.org/abs/2309.08600`.

[4] Boyi Deng, Yu Wan, Baosong Yang, Yidan Zhang, and Fuli Feng. Unveiling language-specific features in large language models via sparse autoencoders. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4563–4608, Vienna, Austria,

July 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.acl-long.229. URL https://aclanthology.org/2025.acl-long.229/.

[5] Sumanth Doddapaneni, Rahul Aralikatte, Gowtham Ramesh, Shreya Goyal, Mitesh M. Khapra, Anoop Kunchukuttan, and Pratyush Kumar. Towards leaving no Indic language behind: Building monolingual corpora, benchmark and models for Indic languages. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12402–12426, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-long.693. URL https://aclanthology.org/2023.acl-long.693/.

[6] Joshua Engels, Logan Riggs Smith, and Max Tegmark. Decomposing the dark matter of sparse autoencoders. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. URL https://openreview.net/forum?id=sXq3Wb3vef.

[7] Jiahai Feng and Jacob Steinhardt. How do language models bind entities in context? In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL https://openreview.net/forum?id=zb3b6oKO77.

[8] Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, et al. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*, 2024. URL https://arxiv.org/abs/2408.00118.

[9] John Hewitt and Percy Liang. Designing and interpreting probes with control tasks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2733–2743, Hong Kong, China, November 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1275. URL https://aclanthology.org/D19-1275/.

[10] Tom Lieberum, Senthooran Rajamanoharan, Arthur Conmy, Lewis Smith, Nicolas Sonnerat, Vikrant Varma, János Kramár, Anca Dragan, Rohin Shah, and Neel Nanda. Gemma Scope: Open sparse autoencoders everywhere all at once on Gemma 2. In *Proceedings of the 7th BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP*, pages 278–300, Miami, Florida, US, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.blackboxnlp-1.19. URL https://aclanthology.org/2024.blackboxnlp-1.19/.

[11] Samuel Marks, Can Rager, Eric J. Michaud, Yonatan Belinkov, David Bau, and Aaron Mueller. Sparse feature circuits: Discovering and editing interpretable causal graphs in language models. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. URL https://openreview.net/forum?id=I4e82CIDxv.

[12] Guilherme Penedo, Hynek Kydlíček, Vinko Sabolčec, Bettina Messmer, Negar Foroutan, Amir Hossein Kargaran, Colin Raffel, Martin Jaggi, Leandro Von Werra, and Thomas Wolf. FineWeb2: One pipeline to scale them all — adapting pre-training data processing to every language. In *Second Conference on Language Modeling (COLM)*, 2025. URL https://openreview.net/forum?id=jnRBe6zatP.

[13] Senthooran Rajamanoharan, Tom Lieberum, Nicolas Sonnerat, Arthur Conmy, Vikrant Varma, János Kramár, and Neel Nanda. Jumping ahead: Improving reconstruction fidelity with JumpReLU sparse autoencoders. *arXiv preprint arXiv:2407.14435*, 2024. URL https://arxiv.org/abs/2407.14435.

[14] Kevin Ro Wang, Alexandre Variengien, Arthur Conmy, Buck Shlegeris, and Jacob Steinhardt. Interpretability in the wild: a circuit for indirect object identification in GPT-2 small. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023. URL https://openreview.net/forum?id=NpsVSN6o4ul.

# Appendices

The original plan and every detail of the pilot study

## A The original position

PRISM (*Probing Residual Information and State Manifolds*) was designed to test a strong version of the "dark matter" idea: that SAEs are blind to the step-by-step state a model computes, and that this state therefore lives in the error $\varepsilon$. It made three hypotheses, each tied to one experiment.

| | Hypothesis | Experiment | Prediction if true |
|---|---|---|---|
| H1 | The error carries the task state | *Causal split:* swap $\hat{x}$ or $\varepsilon$ from the clean prompt into the corrupt one | swapping $\varepsilon$ brings back more of the answer than swapping $\hat{x}$ |
| H2 | The state forms a simple, low-dimensional shape inside the error | *Geometry:* spectra of $\varepsilon$, linear probes for the state, low-dimensional projections | a probe reads the state from $\varepsilon$ better than from $\hat{x}$ and better than from the prompt's words |
| H3 | A few error directions repair the SAE | *Hybrid repair:* $\hat{x}$ plus the clean error along its top $r$ directions | a small $r$ closes most of the gap; random directions do not |

**Original tasks.** Three algorithmic tasks were chosen because their answer depends on a hidden running state: a sum with a remainder (modular arithmetic), the depth of nested brackets, and the size of a graph reachable from a node, plus the names task as a control. All share one format: a clean and a corrupt prompt that differ in one step, with a one-token answer.

**Why they were dropped.** The first overnight run (October 19) kept no usable pairs. A capability study (October 20; 128 pairs per cell, 0 to 8 worked examples, several difficulty levels) showed that gemma-2-2b does no better than simple shortcuts on all three: at the configured difficulty, accuracy was 13% vs a 21% shortcut baseline (remainder sums), 41% vs 41% (bracket depth) and 30% vs 48% (graph size), and almost no pairs survived the filter. Making the tasks easier did not help (remainder sums and bracket depth stayed flat; graph size improved but never became usable). A positive control (the names task, 100% with 2 examples) confirmed the pipeline was sound. Four replacement tasks passed (Figure 2).

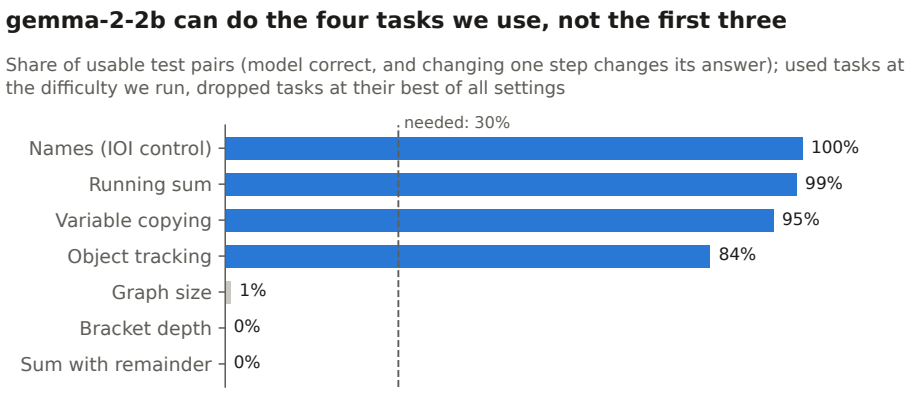



**Figure 2:** Capability check: share of usable pairs per task (30% needed). Used tasks at the difficulty we run, dropped tasks at their best setting.

## B The pilot study

### B.1 Setup

| | |
|---|---|
| Model | gemma-2-2b: 26 layers, internal signal of 2304 numbers per token, bf16 precision |
| SAEs | Gemma Scope residual-stream JumpReLU SAEs [10, 13] (`gemma-scope-2b-pt-res-canonical`), widths 16k, 32k, 65k, 131k, 262k |
| SAE quality | the 16k SAEs recover 98% to 99% of the language-modelling loss on web text (layers 10 to 18) |
| Pairs | 300 training and 150 test pairs per task and setting, prompt-disjoint; all fitting on training pairs only |
| Filter | a pair is kept only if the model answers the clean prompt correctly and the one-step change moves its answer |
| Statistics | 95% paired bootstrap confidence intervals |
| Code checks | 78 unit tests; matches Hugging Face's model (same top answer on 95% of prompts); correct on 6 of 7 sanity prompts |
| Compute | one 8 GB laptop GPU (RTX 5060) |

### B.2 Stages

| Date | Stage | What ran | Outcome |
|---|---|---|---|
| Oct 19 | 0. Overnight run | original three tasks, all three experiments | no usable pairs; no results |
| Oct 20 | 1. Checks | code checked against Hugging Face; capability study of 7 tasks | original tasks dropped; four tasks chosen |
| Oct 25 | 2. Quick pilot | 24 min; 160/80 pairs; layers 6, 12, 18; 16k SAEs | H1 rejected; a small, low-dimensional gap in object tracking and the running sum |
| Oct 26 | 3. Main pilot | three runs, 6.3 h: causal split at layers 4 to 20; repair and specificity at layers 10 to 18; SAE sizes 16k to 262k at layer 12 | results below |

The quick pilot and the main runs agree: object tracking at layer 12 gave 75% vs 74% (features) and 61% vs 62% (leftovers), and one direction closed 51% vs 52% of the gap; the running sum's gap at layer 18 was 12% vs 13%.

### B.3 Measures

**Share of the answer brought back (NLDR).** With $D$ = the model's logit for the clean answer minus its logit for the corrupt answer, NLDR $= (D_{\text{patched}} - D_{\text{corrupt}})/(D_{\text{clean}} - D_{\text{corrupt}})$, averaged over test pairs: 0 = nothing restored, 1 = fully restored. **Swap conditions:** full clean signal (control, must give 1); $\hat{x}$ only (clean $\hat{x}$, corrupt $\varepsilon$); $\varepsilon$ only (clean $\varepsilon$, corrupt $\hat{x}$); noise (corrupt $\hat{x}$ plus random noise of the same size as $\varepsilon$, which erases the corrupt error). **Positions:** swapping from the first changed token to the end (used for all results) or the last token only (Trap 1). **Repair:** $\hat{x}$ plus the clean error projected on $r$ directions $U_r$: $\hat{x} + U_r U_r^\top \varepsilon$. Directions come from the task's own training errors (*unsupervised*: top singular vectors; *supervised*: directions separating the answer classes), from ordinary web text, from another task (*transfer*), or at random. **Gap** $= 1 -$ NLDR($\hat{x}$ only); **share of the gap closed** = (repair $-$ $\hat{x}$ only)/gap. **Probes:** linear classifiers trained to read the answer state, with a baseline probe that sees only the prompt's words.

## B.4 Results

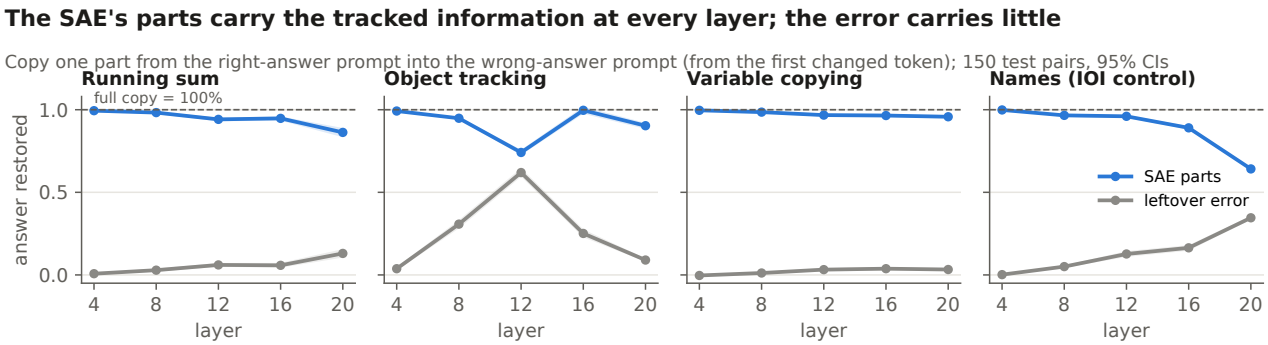



**Figure 3:** Causal split, four tasks, layers 4 to 20 (from the first changed token, 150 test pairs, 95% CIs).

| % brought back: features / leftovers | layer 4 | layer 8 | layer 12 | layer 16 | layer 20 |
|---|---|---|---|---|---|
| Running sum | 99 / 1 | 98 / 3 | 94 / 6 | 95 / 6 | 86 / 13 |
| Object tracking | 99 / 4 | 95 / 31 | 74 / 62 | 100 / 25 | 90 / 9 |
| Variable copying | 100 / 0 | 99 / 1 | 97 / 3 | 96 / 4 | 96 / 3 |
| Names (control) | 100 / 0 | 97 / 5 | 96 / 13 | 89 / 16 | 64 / 35 |

For object tracking the leftovers' score is inflated: replacing the leftovers with noise alone already gives 28% to 58% at layers 8 to 16 (51% at layer 12), because it erases the corrupt prompt's error; the clean error adds about 11 points at layer 12.

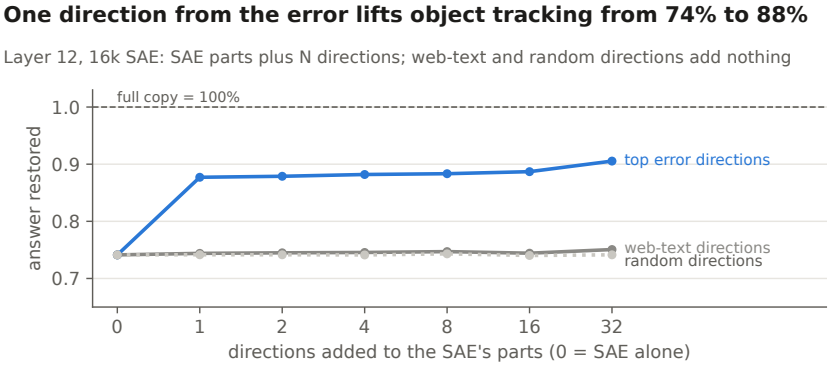



**Figure 4:** Repair for object tracking at layer 12: $\hat{x}$ plus $N$ directions.

| % brought back, $N$ directions added | 0 | 1 | 2 | 4 | 8 | 16 | 32 |
|---|---|---|---|---|---|---|---|
| top directions of the error | 74 | 88 | 88 | 88 | 88 | 89 | 91 |
| web-text directions | 74 | 74 | 74 | 75 | 75 | 74 | 75 |
| random directions | 74 | 74 | 74 | 74 | 74 | 74 | 74 |

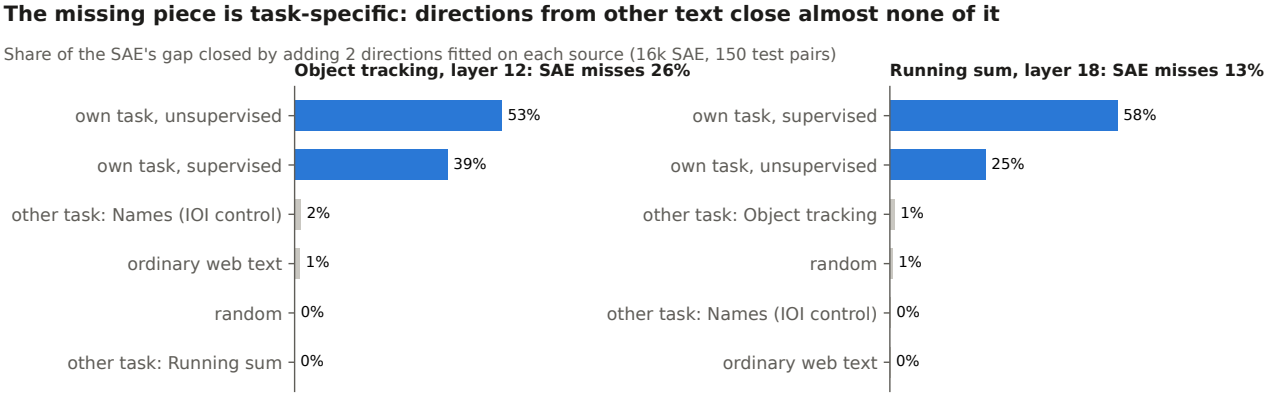



**Figure 5:** Share of the gap closed by 2 directions from each source (16k SAE).

| Task (gap) | Source of the 2 directions | % of gap closed |
|---|---|---|
| Object tracking, layer 12 (26%) | own task, unsupervised / supervised | 53 / 39 |
| | other task: names / running sum | 2 / −1 |
| | ordinary web text / random | 1 / 0 |
| Running sum, layer 18 (13%) | own task, supervised / unsupervised | 58 / 25 |
| | other task: object tracking / names | 1 / 0 |
| | ordinary web text / random | 0 / 1 |

The names control behaves differently: it loses 24% at layer 18 (36% at layer 20), and about 32 own-task directions are needed to close half of that, so its loss is spread out rather than concentrated.

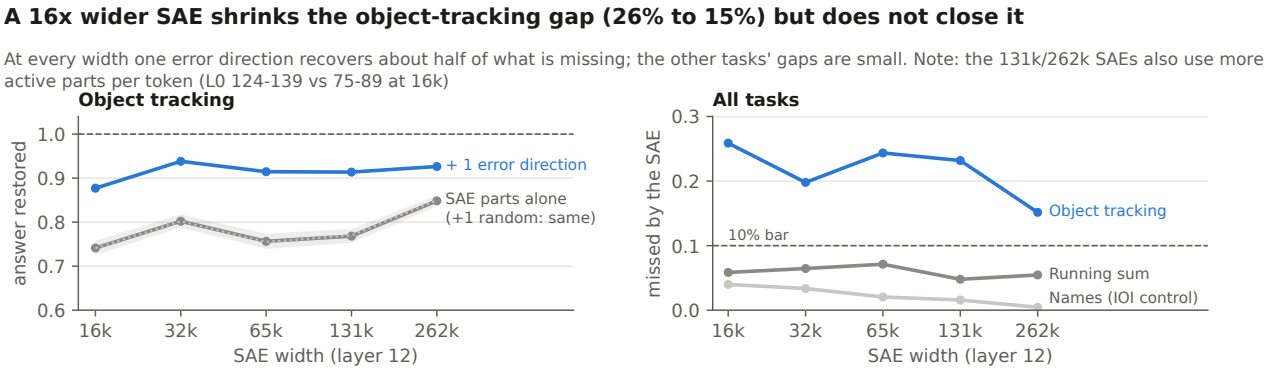



**Figure 6:** SAE size at layer 12.

| Task | Width | features [95% CI] | gap | +1 error dir. | +1 random | active |
|---|---|---|---|---|---|---|
| Object tracking | 16k | 74 [73, 76] | 26 | 88 | 74 | 78 |
| | 32k | 80 [79, 82] | 20 | 94 | 80 | 75 |
| | 65k | 76 [74, 77] | 24 | 91 | 76 | 76 |
| | 131k | 77 [75, 78] | 23 | 91 | 77 | 131 |
| | 262k | 85 [84, 86] | 15 | 93 | 85 | 124 |
| Running sum | 16k / 262k | 94 / 95 | 6 / 5 | 94 / 94 | 94 / 95 | 80 / 129 |
| Names (control) | 16k / 262k | 96 / 100 | 4 / 0 | 96 / 100 | 96 / 100 | 89 / 139 |

Across all widths the running sum's gap stays between 5% and 7% and the names gap at or below 4%.

### B.5 Three testing traps

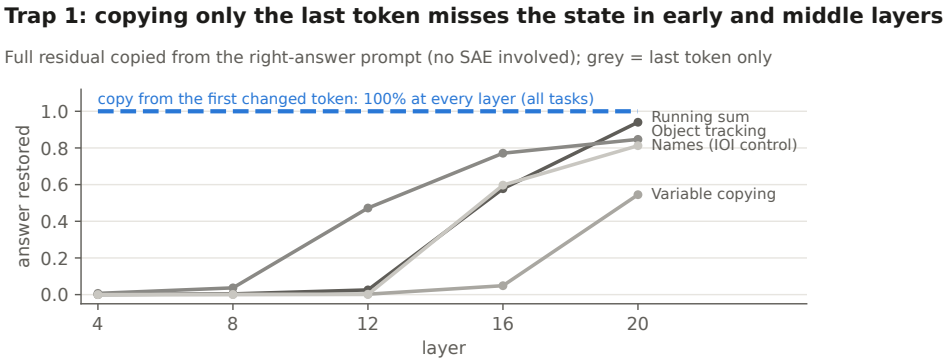



**Figure 7:** Trap 1: swapping the full signal at the last token only vs from the first changed token.

**Trap 1, swapping only the last token.** In early and middle layers the state still sits on earlier tokens, so swapping only the last token restores little even with the *full* signal; from the first changed token it restores 100% at every task and layer. % restored, last token only, layers 4/8/12/16/20: running sum 0/0/3/58/94; object tracking 1/4/47/77/85; variable copying 0/0/0/5/55; names 0/0/0/60/81.

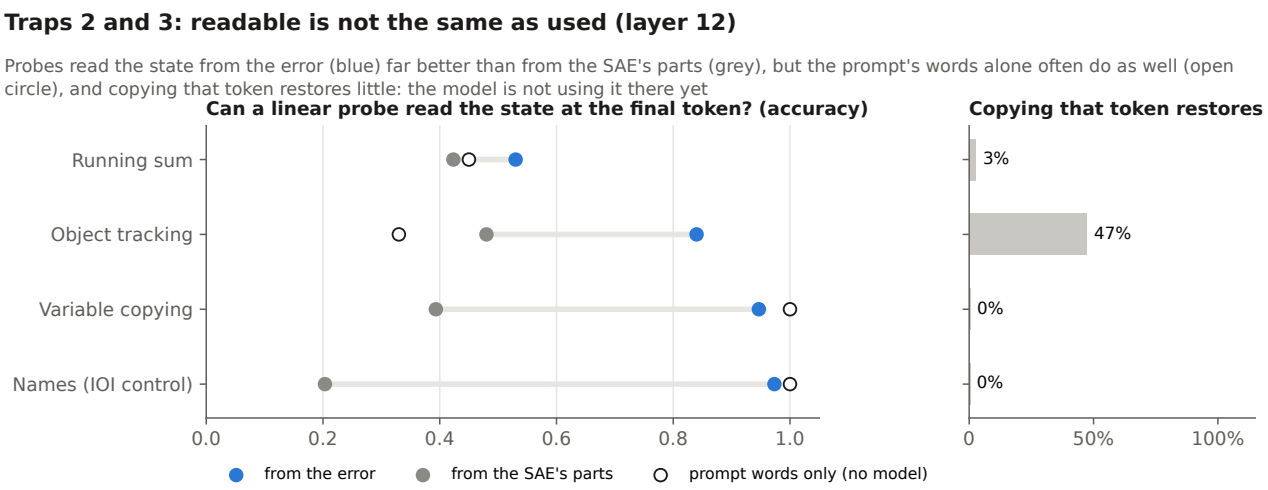



**Figure 8:** Traps 2 and 3 at layer 12.

| Layer 12 | probe on $\varepsilon$ | probe on $\hat{x}$ | words only | chance | last-token swap restores |
|---|---|---|---|---|---|
| Running sum | 53 | 42 | 45 | 25 | 3 |
| Object tracking | 84 | 48 | 33 | 18 | 47 |
| Variable copying | 95 | 39 | 100 | 11 | 0 |
| Names (control) | 97 | 20 | 100 | 11 | 0 |

**Trap 2, the prompt gives it away:** a probe that sees only the prompt's words scores 100% on variable copying and names, so high probe scores there prove nothing [9]. **Trap 3, readable is not used:** probes read the state far better from $\varepsilon$ than from $\hat{x}$, yet swapping that token restores only 0% to 47%: the model is not using it there yet.

### B.6 Verdict on the original hypotheses

| | |
|---|---|
| H1 (error carries the state) | **Rejected** for every task: $\hat{x}$ beats $\varepsilon$ at every layer. |
| H2 (simple shape in the error) | **Not supported as stated**: the state is readable from $\varepsilon$ but not used there, and for two tasks the prompt's words alone give it away. |
| H3 (a few directions repair) | **Supported, narrowly**: 1 direction closes about half of object tracking's gap, 2 directions 58% of the running sum's; the directions are task-specific. This became the project's claim. |

### B.7 Caveats

- One model (gemma-2-2b), one SAE family (Gemma Scope), synthetic tasks; object tracking keeps only 27% of candidate pairs after filtering, so its numbers are the noisiest.
- The names control's late-layer loss (24% to 36% at layers 18 to 20) needs about 32 directions to recover half: part of the late-layer loss is a general SAE weakness.
- The 131k and 262k Gemma Scope SAEs also use more active features per token (124 to 139 vs 75 to 89 at 16k), so part of the shrinking gap may come from sparsity rather than width.
- The binding explanation is a hypothesis, and the multilingual work is a plan with no results yet.

## C Multilingual design

| ID | Hypothesis | Measure | Supported if |
|---|---|---|---|
| M1 | SAEs explain Indian-language text worse | SAE loss recovered on ordinary text per language | clearly below English (98% to 99%) for Hindi / Tamil |
| M2 | SAEs miss more task state in Indian languages | gap and specificity per language | gap above English with non-overlapping CIs |
| M3 | The missing piece is language-independent | direction fitted on English, applied to Hindi | closes a similar share of the Hindi gap |

**Languages:** English (baseline); Hindi (Devanagari; the best-resourced Indian language, most likely to work); Hinglish (code-mixed Hindi in Latin script; separates language from script); Tamil or Bengali

(another script and family; likely harder for a 2B model); French or Spanish (high-resource non-Indian control, separating "not English" from "Indian and low-resource").

**Templates.** The answer must be the last token, so verb-final Hindi uses a question plus "answer" format. All examples are *illustrative, to be checked by a native speaker and against the tokenizer*:

- Running sum (Hindi): शुरुआत 2 से करें। 3 जोड़ें। 1 जोड़ें। 2 जोड़ें। कुल = → 8
- Object tracking (Hindi): सिक्का डिब्बे B में है। चाबी डिब्बे E में है। चाबी डिब्बे C में जाती है। चाबी किस डिब्बे में है? डिब्बा → C
- Names (Hinglish): `Jab Rahul aur Priya market gaye, Priya ne ek book di. Book kisko mili? Answer:` → Rahul
- Variable copying is code-like and identical in every language: a language-neutral control.

**Constraints:** clean and corrupt prompts need the same number of tokens and a one-token answer; many Hindi names are several tokens, so names are picked from single-token ones (the pipeline already drops and counts violating pairs). **Steps:** (1) capability check per language (usable pairs $\geq 30\%$); (2) SAE fidelity on IndicCorp v2 and FineWeb-2 text (M1); (3) swap test, gap, repair and specificity per language (M2); (4) cross-lingual transfer of the English direction (M3). **Fallbacks:** if Tamil or Bengali fail the capability check, report that and keep Hindi and Hinglish; if names are too constrained, use letters or digits as answers.

## D Glossary

| | |
|---|---|
| Internal signal (residual stream) | the model's running internal representation at one layer: 2304 numbers per token |
| Sparse autoencoder (SAE) | a tool that rewrites the internal signal as a few active, human-readable features |
| Readable part $\hat{x}$ | the SAE's reconstruction from its features |
| Error, leftovers $\varepsilon$ | $x - \hat{x}$: what the SAE cannot explain ("dark matter") |
| Width | the number of features an SAE can choose from (16k to 262k) |
| Clean / corrupt prompts | two prompts that differ in one step, so the right answer differs |
| Swap test (patching) | run the corrupt prompt but copy part of the signal from the clean prompt at one layer |
| NLDR | share of the right answer brought back by a swap (0 to 1) |
| Gap | the share the SAE's features miss: 1− NLDR of swapping $\hat{x}$ only |
| Direction | a single pattern in the 2304 numbers; "$N$ directions" = an $N$-dimensional slice of the signal |
| Probe | a simple classifier trained to read a value (e.g. the box) from the signal |
| Binding | how a model links an entity to its attribute (an object to its box) |